가상장치 실습 안내 — 자기 PC 코드로 PLC와 로봇 연결하기

이 실습은 학생이 고친 학생코드/picknplace_full/sequence.py를 가짜 PLC와 가짜 로봇에 연결한다. 세 프로그램이 따로 실행되므로 PC의 명령, PLC의 접점·타이머, 로봇의 DI/DO·움직임을 나란히 확인할 수 있다. 제공 Modbus TCP 연결을 사용한다. 일반 TCP 통신이나 실제 HCR 직접 제어 프로토콜을 새로 배우는 단원은 아니다.

1. 두 실행 방법을 구분한다

실행 방법	용도	확인할 화면
lesson_ui.py 또는 --target model	빠른 함수 실습과 고장 주입. PC 안의 모형 한 개	PC 화면 8877
가짜 PLC + 가짜 로봇 + --target tcp:127.0.0.1:5020	자기 순서 코드와 장치 간 신호를 연결하는 통합 실습	PC 8877, PLC 8878, 로봇 8879

두 방법의 PC 앱을 동시에 켜지 않는다. 둘 다 8877을 쓰기 때문이다. lesson_scenarios.py는 실행할 때마다 별도 내부 모형을 만든다. 그 고장은 이미 켜 둔 가짜 PLC·로봇 화면에 적용되지 않는다.

2. 같은 코드 폴더에서 터미널 세 개를 연다

학생용 ZIP을 풀고 학생코드 폴더를 연다. requirements.txt, fake_plc, fake_robot, picknplace_full, ladder, robot이 나란히 보이는 위치다. 이 폴더에서 PowerShell 세 개를 연다. 각 터미널에서 Get-Location으로 **세 경로가 같은지** 확인한다. 강사 시연은 같은 방법으로 강사코드 폴더에서 한다. 외부 원본 저장소에서 실행하면 학생 수정본을 시험한 것이 아니다.

처음 한 번, 같은 Python에 필요한 패키지를 준비한다.

```
python --version
python -m pip install -r requirements.txt
```

Python 3.11 이상을 사용한다. 내장 로봇과 브라우저 스크립트 실습에는 Node 설치가 필요 없다. 브라우저용 라이브러리는 제공 폴더에 포함된다. 아래 명령의 -B는 캐시 생성을 막고 -X utf8은 한글 출력을 맞춘다.

터미널 1 — PLC를 먼저 켠다

```
python -B -X utf8 -m fake_plc ladder/PICKNPLACE_FULL.xgwx --modbus-port 5020 --wiring-port 5021 --http-port 8878
```

http://127.0.0.1:8878을 연다. 파일이 PICKNPLACE_FULL.xgwx인지, RUN인지 확인한다. 5020은 PC의 Modbus 연결, 5021은 가짜 로봇과 배선 신호를 교환하는 내부 연결이다. 이 포트 번호를 실제 HCR의 접속 포트로 설명하지 않는다.

PLC는 RUN 직후 그리퍼 열기 타이머를 약 3초 진행한다. **0002가 보이거나 PC 시작 버튼이 켜져도 이 준비가 끝난 뜻은 아니다.** 이 구간에는 1111·9991·9992·9994·3333이 거부될 수 있다. 먼저 3초 이상 기다리고 PLC에서 G_OPENING인 M00418이 꺼졌는지 확인한다. T0300은 그리퍼 타이머다. 완료 후 타이머가 다시 0으로 돌아갈 수 있으므로 3000ms가 계속 남을 것으로 기대하지 않는다.

터미널 2 — 로봇을 연결한다

```
python -B -X utf8 -m fake_robot --plc-host 127.0.0.1 --wiring-port 5021 --http-port 8879
```

http://127.0.0.1:8879를 연다. PLC 연결 없음이 해소되고 로봇 상태·DI/DO 표시등이 보이는지 확인한다. 처음은 내장 스크립트 (DI0 → PICK, DI1 → HOME)다. 내장 모드로 연결부터 확인한 다음 5절의 파일 실행 모드로 넘어간다.

터미널 3 — 자기 프로그램을 연결한다

```
python -B -X utf8 -m picknplace_full --target tcp:127.0.0.1:5020 --http-port 8877
```

http://127.0.0.1:8877을 연다. 화면의 대상이 tcp:127.0.0.1:5020인지 확인한다. model이면 다른 실행을 보고 있다. URL은 모두 localhost 대신 위에 적은 127.0.0.1을 사용한다. 가짜 장치 웹 서버는 허용 주소를 검사한다.

아무 버튼도 누르지 않고 PC 상태 숫자·해석·시각, PLC D01000·D01001, 로봇 DO0·DO1을 기록한다. A/B 미완성 상태에서도 관측과 제공된 개별 조작은 가능하다. 자동 한 사이클은 A와 B 정상 순서를 완성한 뒤 실행한다. C의 정상 대기는 제공되어 있어 C의 시간 초과 과제를 끝내기 전에도 정상 경로를 시험할 수 있다.

3. 세 화면을 읽는 순서

화면	볼 것	그 값으로 말할 수 있는 것
PC 8877	대상, 원시 상태, 단계, 명령 결과, 로그 시각	자기 코드가 무엇을 요청했고 어떤 새 상태를 읽었는가
PLC 8878	래더 접점·코일 표시, P/M/T/D, Modbus 로그	요청을 받아들이는 조건, 출력과 타이머가 바뀌었는가
로봇 8879	DI0/DI1/DI3, DO0/DO1, 3D와 실행 로그	요청을 받았는가, 움직였는가, 정지·응답 신호는 무엇인가

PLC의 파란 접점·비교식·코일 표시는 가상 스캔의 현재 판정이다. P는 입출력, M은 내부 상태, T는 타이머, D는 숫자 데이터다. 기본 시험에서는 P/M 강제 입력을 쓰지 않는다. 강제 값은 로봇 배선 신호보다 우선할 수 있어 관측을 바꾼다. 이미 강제했다면 해당 항목을 해제하고 기록한다.

이동1을 한 번 요청하고 다음 연결을 찾아라. PC 요청 9991 → PLC P00044 → 로봇 DI0 → 로봇 DO0 → PLC P00004 → D01000 → PC의 새 관측이다. 이동2는 P00045/DI1을 쓴다. 정지는 P00047/DI3, 비정상 정지 응답은 DO1/P00005다. 이것은 가상 배선의 계약이며 실제 전기 사양 확인과 구별한다.

DO0은 목표에 도달했다고 즉시 꺼지지 않는다. 목표 도달 **그리고** PLC 요청 DI0·DI1이 모두 OFF라는 두 조건 뒤에 꺼진다. PLC가 DO0 ON을 보고 요청을 내리고, 로봇이 요청 해제를 확인하는 순서를 핸드셰이

크라고 부른다. 이미 같은 위치에 있어서 이동이 거의 없어도 요청과 응답을 놓치지 않게 한다. DO0 ON만으로 매 순간 팔이 움직인다고 말하지 않는다. 빠른 변화는 화면에서 건너뛰어 보일 수 있으므로 로그·관측 시각을 함께 남기고 못 본 변화는 미확인으로 적는다.

4. 자기 순서 함수를 적용한다

1. 학생 폴더에서 A와 B 정상 검사를 실행한다. python -B -X utf8 check_student.py --stage A, 이어서 python -B -X utf8 check_student.py --stage B --part normal을 실행한다.
2. 준비가 끝난 같은 세 화면에서 PC의 시작을 누른다. 1111 → 9991 → 9992 요청, 이동1 → 닫힘 → 이동2 → 열림의 관측을 기록한다. 자동 운전의 그리퍼는 PLC가 처리하므로 PC 코드에 3333을 추가하지 않는다.
3. 학생 sequence.py를 고쳤으면 작업을 마치고 터미널 3의 PC 앱을 Ctrl+C로 종료한 뒤 같은 명령으로 다시 켠다. PLC·로봇까지 무조건 초기화하지 않는다. PC 재시작은 진행 중 작업의 복구나 장치 정지가 아니다.
4. 중단된 실험을 초기 상태로 다시 만드는 경우는 이전 로그를 먼저 저장하고 **새 시험**이라고 기록한다. PLC STOP/RUN은 첫 스캔을 다시 수행하며 P/M/T를 초기화하고 D는 유지한다. 파일을 다시 열면 STOP 상태가 되고 D도 지워진다. 이를 9994 접수로 복구한 경우와 혼동하지 않는다.

검사 통과는 작은 조건 시험이다. 같은 학생 폴더에서 세 프로그램을 연결한 결과가 실제 통합 근거다. 이 경우에도 실제 PLC·HCR 성공은 아니다.

5. 제공 로봇 파일을 선택하고 다시 읽는다

실습은 제공 로봇 Thread를 사용한다. 학생에게 제어 스레드를 새로 구현시키지 않는다.

1. PC 작업이 끝나고 로봇 요청·출력이 정리된 상태에서 8879 화면을 연다. 스크립트 실행용 탭은 하나만 둔다.
2. 학생 스크립트 (Rodi Script 파일)의 파일 선택에서 같은 코드 폴더의 robot/PICKNPLACE_FULL_main.js를 고른다. 파일 선택은 내용을 읽고 스크립트 실행을 시작한다. 화면에 선택한 파일명이 보이는지와 로그의 오류를 확인한다.
3. 이 모드는 브라우저 안 실행기가 로봇 프로그램을 실행한다. 탭을 계속 열어 두고 화면을 볼 수 있게 둔다. 탭을 닫거나 실행 응답이 끊기면 DO가 내려갈 수 있다. 그때 나온 완료 표시를 정상 완료 시험으로 채택하지 않는다.
4. .js를 수정해 저장했다면 **파일을 다시 선택한다.** 다시 실행 (읽어 둔 내용)은 이전에 읽은 문자열을 다시 돌릴 뿐 디스크 파일을 새로 읽지 않는다.
5. 내장 모드의 HOME/PICK 입력과 적용 (다음 이동부터)는 내장 로봇 설정이다. 선택한 파일 안 HOME/PICK 변수를 바꾸지 않는다.

파일 모드의 ES5 제한은 **가짜 실행기의 지원 범위**다. var, function, while, sleep 등을 쓰며 let, const, 화살표 함수는 이 실행기에서 오류가 난다. 이를 실제 HCR 언어 전체의 지원 규격으로 일반화하지 않는다. 실제 API 중 가짜 실행기가 구현하지 않은 명령도 있다. 오류의 함수명·줄을 남기고 지원 범위를 확인한다.

제공 HOME/PICK은 가상 시연용 예시 좌표다. 실제 로봇에 그대로 옮기지 않는다. 3D의 치수·운동은 근사이며 충돌·축 한계·실제 정지 거리 검증이 아니다. Thread 안 moveStop과 전역 변수 공유는 실제 장치 확인이 남아 있다.

6. 즉시 정지를 누른 시점에 따른 세 시험

기능 정지 요청 9995는 안전 비상정지 회로가 아니다. 먼저 정상 한 사이클을 확인한다. 이동 중 시험에서는 **PC의 0001만 보고 누르지 않는다.** 0001에는 PLC의 요청 대기 구간도 포함된다. 로봇 DO0 ON·3D 이동을 확인하고, PLC 화면에서 해당 이동의 RUN1(M00414) 또는 RUN2(M00415)가 ON인지 함께 확인한 뒤 누른다. RUN1·RUN2는 PLC가 로봇 시작 신호를 받아 기억하는 내부 비트이며 PLC 운전 모드 RUN과 다르다. 로봇 DO0만 켜지고 PLC가 아직 시작을 확인하지 못한 순간에 누르면 요청 취소와 겹쳐 자동 그리퍼가 시작되지 않거나 PC 대기가 시간 초과할 수 있다. 두 확인을 기록하지 못했다면 같은 조건에서 다시 시험한다.

위 조건은 세 화면을 보며 수동으로 정지하는 시험의 조건이다. 별도 내부 모형의 lesson_scenarios.py --case pause는 제공 코드가 정지 입력 시점을 통제하는 독립 사례이며, 그 출력으로 세 프로그램의 PLC RUN1·RUN2를 직접 확인했다고 기록하지 않는다.

누른 구간	재개 전 예상	재개 뒤 확인
2단계 이동1, DO0·움직임·PLC RUN1 확인 뒤	DO0 유지, DO1 ON, robot=2. PC는 다음 완료를 기다림	9993 뒤 남은 이동, 닫힘, 후속 순서 진행
3단계 그리퍼 닫힘 기다림	닫힘 타이머는 끝날 수 있음. _accept가 새 robot=1을 기다리므로 다음 9992는 아직 안 나감	9993 뒤 robot=1 새 관측, 그 다음 9992 접수
5단계 그리퍼 열림 기다림	열림이 끝나면 PC 결과는 완료일 수 있음. 재개 없이도 robot=2가 남음	완료와 별도로 9993을 보내 기능 정지 해제 확인

3단계 시험은 PC 단계명과 PLC M00417을, 5단계는 단계명과 M00418을 함께 확인하고 누른다. 화면이 4단계로 바뀌었더라도 다음 전송 전 대기 중일 수 있으므로 9992 로그가 있는지 따로 본다. 결과가 예상과 다르면 누른 시점과 실제 신호를 기록하고 준비 후 재시험한다. 수업 기본 대기의 move_timeout은 120초이며 정지한 시간도 해당 대기에 포함된다. 개별 스크립트 버튼 시험에는 자동 순서의 이 대기를 그대로 적용하지 않는다.

5단계 차이를 짧게 재현하는 별도 내부 모형 명령도 있다. A/B를 완성한 학생 폴더에서 실행한다.

```
python -B -X utf8 lesson_scenarios.py --case final_open_estop
```

예상 핵심은 result: 완료, resume_sent: false, robot_abnormal_remains: true다. 이 명령은 켜 둔 세 프로그램의 상태를 바꾸지 않는다. 같은 현상을 독립 조건에서 재현한 결과로 기록한다.

7. 막혔을 때 확인 순서

현상	먼저 확인할 것과 대응
모듈이나 래더 파일을 찾지 못함	세 터미널의 Get-Location, 코드 폴더 구조, requirements 설치에 쓴 Python을 대조
포트 사용 중	자기 이전 앱의 터미널에서 Ctrl+C. 8877을 임의로 8878로 바꾸지 않음: 8878은 PLC 화면
로봇에 PLC 연결 없음	터미널 1 실행·5021 설정·터미널 2 오류 확인. 연결은 주기적으로 재시도됨

PLC STOP 또는 빨간 이유 표시	파일 경로·지원 명령·로드 오류를 확인. 오류를 지우지 않고 기록. RUN을 반복 눌러 해결했다고 하지 않음
0002인데 시작 거부	첫 RUN 3초 준비, M00418/M00417, 최신 PLC 거부 상태 확인. 준비 완료 전에 자동 재전송하지 않음
PC만 정상인데 3D가 안 움직임	PC 대상이 model인지, 요청 DI가 오는지, 로봇 모드·선택 파일·DO 로그를 차례로 대조
파일을 고쳤는데 그대로임	PC Python 수정은 PC 앱 재실행, 로봇 .js 수정은 파일 재선택. 두 절차를 구분
3D만 보이지 않음	WebGL 오류 확인. DI/DO·상태·로그가 동작하는지 별도로 기록하며 3D 성공이라고 하지 않음
실패 시나리오가 세 화면에 안 보임	lesson_scenarios.py와 lesson_ui.py --fault는 별도 내부 모형. 가상장치 대상 고장 주입으로 설명하지 않음

PLC의 노란 경고는 추정 지원·미검증 요소 등 가상 실행의 한계를 알린다. 세 화면이 움직였다는 사실로 XG5000 프로그램 검사나 실제 PLC 동작이 검증됐다고 하지 않는다.

8. 제출할 근거

학생 기록지에 세 터미널의 폴더·명령, PC 대상, 로봇 모드·파일명, 최초 RUN 준비 확인, 정상 한 사이클, 정지 시점별 결과를 적는다. 실제 관측과 예상을 구분한다. 실패한 조건 하나를 고쳐 같은 조건으로 재시험하고 요청·새 관측·후속 요청 로그를 묶는다. 실행 결과의 종류는 내부 모형, 세 프로그램 가상장치, 실물로 구분한다.

종료는 정상 작업과 신호가 정리된 뒤 PC·로봇·PLC의 각 터미널에서 Ctrl+C로 한다. 강제 종료·PLC 재시작을 기능 정지나 복구 성공으로 기록하지 않는다. 이 안내의 가상장치 성공은 전기 배선, XG5000 실행, 실제 HCR 움직임, 그리퍼 파지 성공을 대신하지 않는다.